

Assignment 1 and 2

Part I: Short Answer Questions

Q.1

Ans:- Difference between DBMS and Traditional File System

DBMS

- ◆ Stores data in a structured manner (tables, rows, columns).
- ◆ Reduces duplication of data.
- ◆ Allows easy searching and querying using SQL.
- ◆ Provides security and access control.
- ◆ Supports multiple users accessing data at the same time.
- ◆ Offers backup and recovery mechanisms.

Traditional File System

- ◆ Stores data in separate files.
- ◆ Data duplication is common.
- ◆ Searching and updating data is more difficult.
- ◆ Limited security features.
- ◆ Concurrent access can lead to inconsistencies.
- ◆ Backup and recovery are usually manual.

Scenario Where DBMS is Better

Banking System

A DBMS is a better choice for a banking system because:

- Customer, account, and transaction data can be stored in one centralized database.
- Multiple customers can perform transactions simultaneously without corrupting data.
- Data remains consistent during operations such as money transfers.
- Security features protect sensitive financial information.
- Backup and recovery help prevent data loss in case of system failures.

Q.2

Ans:- **Weak Entity vs Strong Entity**

Strong Entity

- Can be uniquely identified by its own primary key.
- Does not depend on any other entity for identification.
- Example: Employee with Employee_ID.

Weak Entity

- Cannot be uniquely identified by its own attributes.
- Depends on a strong entity (owner entity) for identification.
- Example: Dependent of an employee. Many employees may have dependents with the same name, so Dependent_Name alone is not enough.

The primary key of a weak entity is formed by combining:

- The primary key of the strong (owner) entity, and
- The weak entity's partial key (also called a discriminator).

Example:

- Employee(Employee_ID)
- Dependent(Dependent_Name)

Primary Key of Dependent = (Employee_ID, Dependent_Name)

The relationship between a weak entity and its owner entity is called an Identifying Relationship. It shows that the weak entity's existence depends on the strong entity.

Q.3

Ans:- The phone number should be modeled as a multivalued attribute.

Reason:

- A simple attribute can store only one value for each entity.
- A composite attribute can be divided into smaller parts (for example, Address → Street, City, State).
- A multivalued attribute can store multiple values for the same entity.

In this case, a student may have more than one phone number, such as 9876543210 or 9123456789

Since a single student can have multiple phone numbers, Phone_Number is a multivalued attribute.

Q.4

Ans:- **Difference Between Candidate Key and Super Key**

Super Key

- Any set of attributes that can uniquely identify a tuple (row).
- It may contain extra attributes that are not necessary for uniqueness.

Candidate Key

- A minimal super key.
- No attribute can be removed while still uniquely identifying a tuple.

Given Relation

Student(roll_no, email, name, branch)

Assume:

- Each roll_no is unique.
- Each email is unique.

Candidate Keys

Since both can uniquely identify a student and are minimal:

- {roll_no}
- {email}

Super Keys (not Candidate Keys)

These also uniquely identify a student, but contain extra attributes:

- {roll_no, name}
- {roll_no, branch}

Q.5

Ans:- Part(1) :- The Referential Integrity Constraint (Foreign Key Constraint) is violated.

Part(2):-

- It ensures that a foreign key value in one table refers to an existing record in another table.
- It prevents orphan records, i.e., records that reference non-existent data.

Part(3):-

The insert operation should be rejected (fail).

The DBMS should generate an error indicating a foreign key/referential integrity violation.

The row should not be inserted until a matching student record exists in the Student table.

Q.6

Ans:-

- A separate relationship table (also called a junction table or bridge table) must be created.
- The primary key is usually the combination of the primary keys of the participating entities.
- Because in an M:N relationship:
 - One student can be related to many courses.
 - One course can be related to many students.
- If we add a course_id column to Student:
 - A student may need multiple course_ids.
- If we add a student_id column to Course:
 - A course may need multiple student_ids.

This would require storing multiple values in a single column, which violates the principles of the relational model. Therefore, a separate relationship table is needed to store all student-course pairs.

Part II: Long Answer Question

Part A – Entity and Relationship Identification

Ans:- **Entities and Key Attributes**

1. Customer:- Customer_ID (Primary Key), Name, Phone, Address
2. Restaurant :-Restaurant_ID (Primary Key), Restaurant_Name, Location
3. Menu_Item:- Item_ID (Primary Key), Item_Name, Price
4. Order:- Order_ID (Primary Key), Order_Date, Total_Amount
5. Order_Item (Associative Entity):- Order_ID (Part of Primary Key, Foreign Key), Item_ID (Part of Primary Key, Foreign Key), Quantity

Relationships and Cardinalities

1. Customer — Places — Order

Cardinality: 1 : N

Reason: One customer can place many orders, but each order is placed by exactly one customer.

2. Restaurant — Receives — Order

Cardinality: 1 : N

Reason: A restaurant can receive many orders, but each order belongs to exactly one restaurant.

3. Restaurant — Offers — Menu_Item

Cardinality: 1 : N

Reason: A restaurant can offer many menu items, and each menu item belongs to one restaurant.

4. Order — Contains — Menu_Item

Cardinality: M : N

Reason: One order can contain multiple menu items, and the same menu item can appear in many different orders.

Part B – Attribute Classification

Ans:- 1. Multivalued Attribute:- Customer Phone Number because A customer may have more than one phone number.

2. Composite Attribute:- Customer Address because an address can be broken down into smaller meaningful parts.

Part C – Relational Schema Design

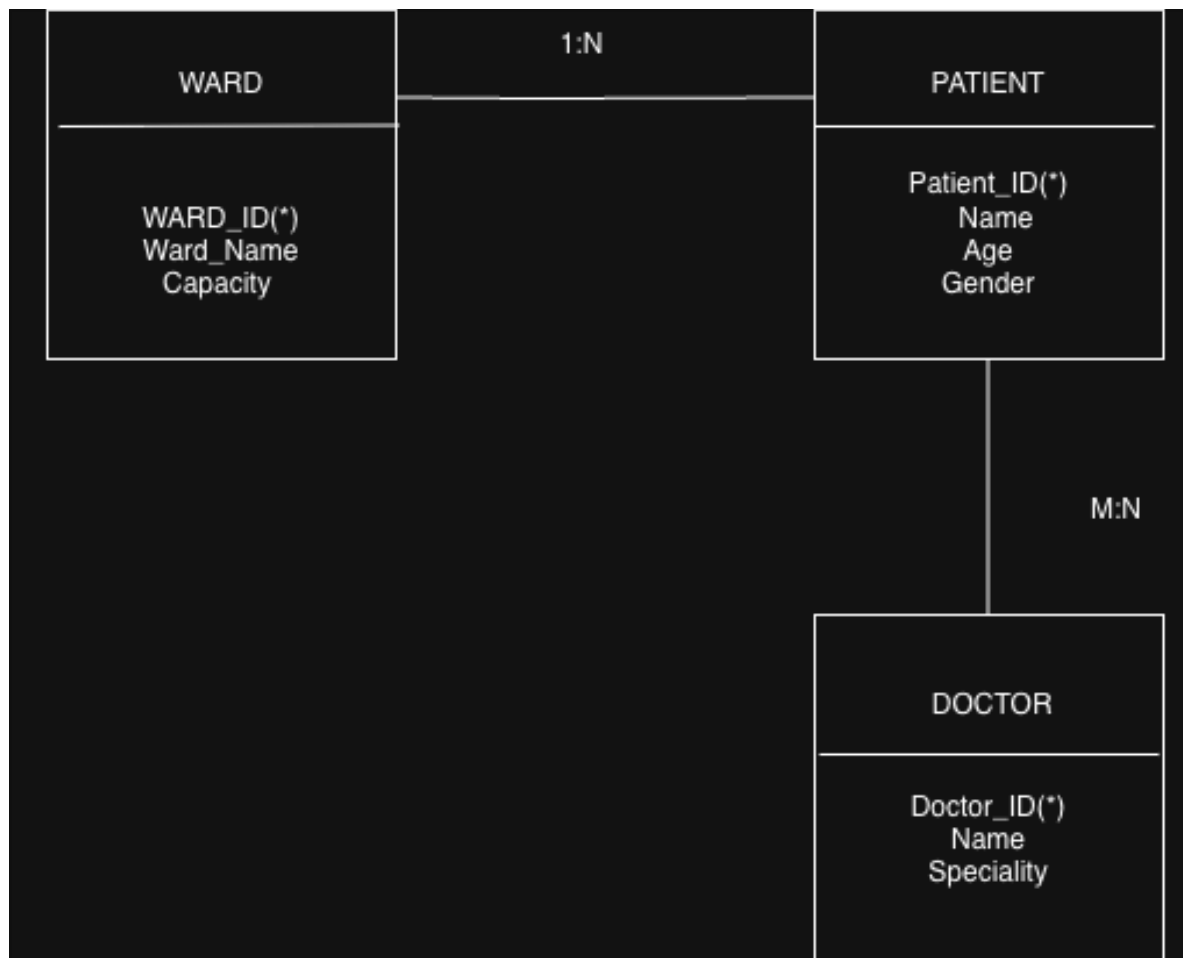
Ans:- Order_Item(Order_ID, Item_ID, Quantity)

- Primary Key: (Order_ID, Item_ID)
- Foreign Keys:
 - Order_ID → Order(Order_ID)
 - Item_ID → Menu_Item(Item_ID)
- Additional Attribute: Quantity (stores the number of units of a menu item in an order)

Reason: Since the relationship between Order and Menu_Item is M:N, a junction table Order_Item is required to represent the relationship.

Part III: ER Diagram Design

Question 1: Hospital ER Diagram



(*) marked represents primary keys.

Question 2: Design Justification

Ans:-

- The Doctor–Patient relationship is many-to-many (M) because a doctor can treat multiple patients, and a patient can be treated by multiple doctors during their stay. To represent this relationship in a relational database, an associative entity called Treatment is created. The schema of this entity is Treatment(Doctor_ID, Patient_ID). Its composite primary key consists of (Doctor_ID, Patient_ID), which uniquely identifies each treatment relationship.
- In the Ward–Patient relationship, the participation of Patient is total because every admitted patient must be assigned to a ward. The participation of Ward is partial because a ward may exist even if no patients are currently admitted to it. In the Doctor–Patient relationship, both Doctor and Patient have partial participation since a doctor may not currently be treating any patient and a patient may not yet have been assigned a doctor.
- One attribute design decision is the use of Address as a composite attribute. Address can be divided into meaningful components such as House Number, Street, City, State, and PIN Code. Modeling it as a

composite attribute improves data organization and allows more efficient searching and reporting based on individual address components.